Basic Selenium Scripting Tutorial

Using Selenium IDE Firefox Add-On v2.9.0

Author: Andrew Chan

Table of Contents

1 TIPS ON PROBING
2 BASIC SELENIUM SCRIPTING
- 2.1 Recording the script
- 2.2 Organizing the script
- 2.3 Testing the recorded script

3 ADDING FLOW CONTROL TO THE SCRIPTS
- 3.1 Download the Flow Control Firefox add-on
- 3.2 Adding Flow Control to the script

4 USING VARIABLES IN YOUR SELENIUM SCRIPT
- 4.1 FOR loop
- 4.2 Return variable

1 TIPS ON PROBING

When creating Selenium scripts:

- minimize the number of steps/transactions.
- make the script as simple as possible.

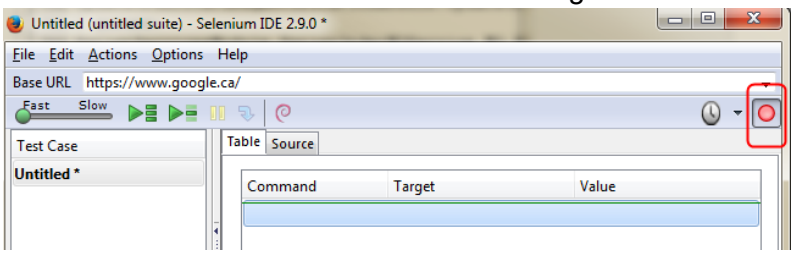
Remember, the idea of probing is to find out if a site works properly. A script should report the problem. It is not meant to be used to diagnose the problem.

2 BASIC SELENIUM SCRIPTING

For basic Selenium scripting (probably covering 90% of the probes), follow the instructions below for recording the scenario. For the purpose of this tutorial, we will create a script that opens the Google homepage, searches for “Selenium IDE” and clicks the Selenium homepage from the results page.

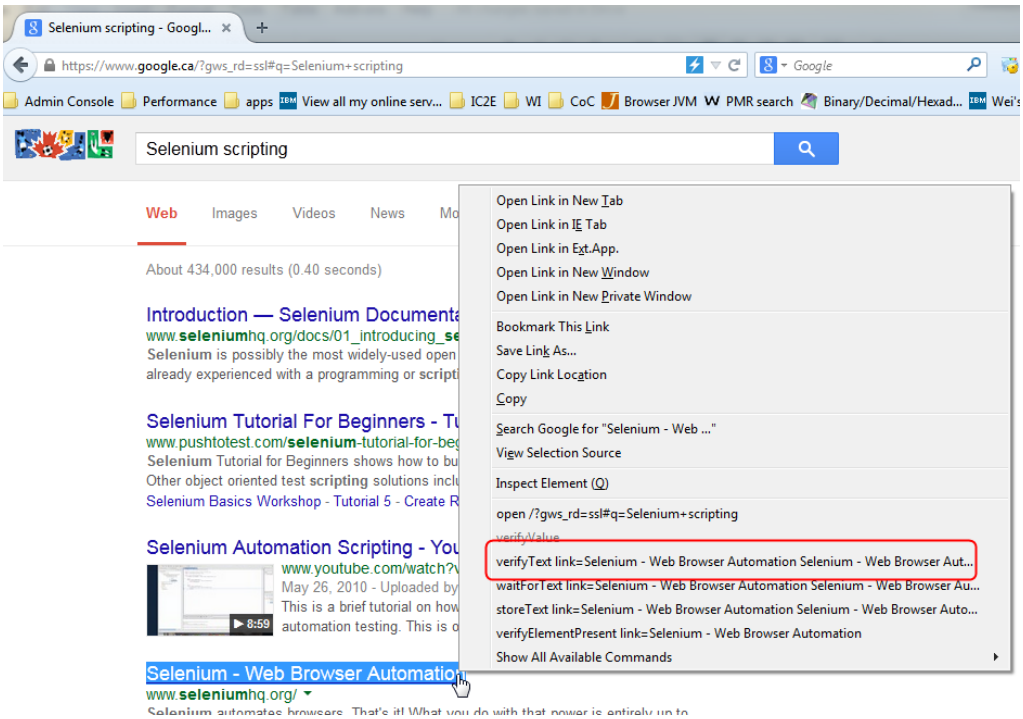
2.1 Recording the script

1) Download and install **Selenium IDE** Firefox plugin from http://www.seleniumhq.org/download/ (v2.9.0 at time of authoring this tutorial).
2) Select “Install now” to install all 5 plugins on the next screen.
3) Launch Selenium IDE from Firefox (FF). Clear out the tabs in FF and click on the red record button to make sure that it’s recording:

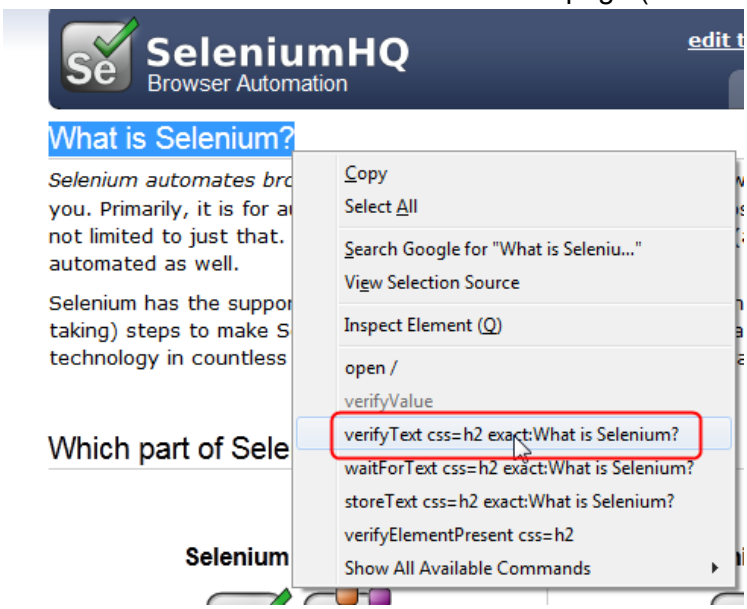


4) In the FF address bar, enter “www.google.com”. In the search box, enter the term “Selenium IDE” and press **Enter** (the result might appear before but we are still going to press enter in this tutorial).
5) You will notice there are some commands already recorded in the Selenium IDE. When you get to the search results page, you want to make sure that the results page contains the Selenium homepage. In FF, highlight the “Selenium - Web Browser Automation” link and right click. In the right click menu, click “verify Text link=Selenium - Web Browser

Automation Selenium….”



6) That should automatically insert a verifyText command into your script to make sure the link exists on the results page. Please note that **verifyText** (or **assertText**) is the single most important Selenium command as you can essentially use this on every page to make sure the scenario is still on track and probe is still functioning normally.
7) Finish the probing scenario by clicking on the “Selenium - Web Browser Automation” link (remember, Selinium should still be recording!)
8) Ensure that the script checks that the correct page loads by verifying that the text “What is Selenium?” exists on the Selenium homepage (same method as step 4):

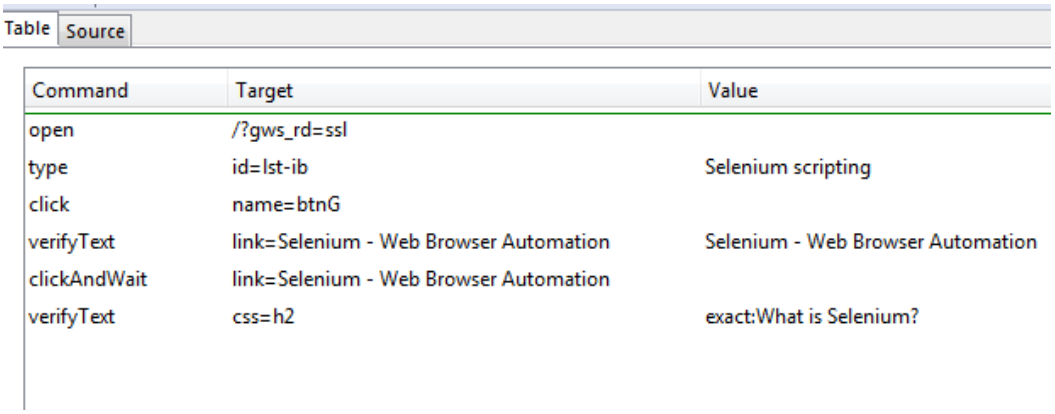


9) There you have finished recording your probing scenario.

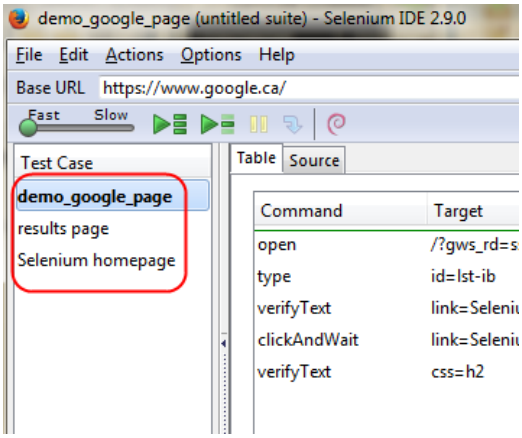
2.2 Organizing the script

In order to best use the Selenium script with the Synthetic Playback agent in Performance Management, the script needs to be rearranged in such a way that each test case represent one page. We will continue to use the script created above as an example.

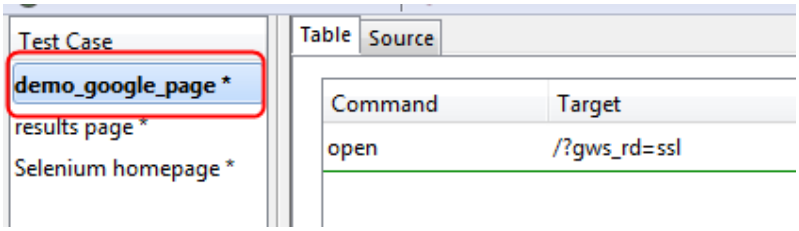
1) Before any rearrangement, stop the recording. The recorded script should contain all commands under one 'Untitled' Test Case.

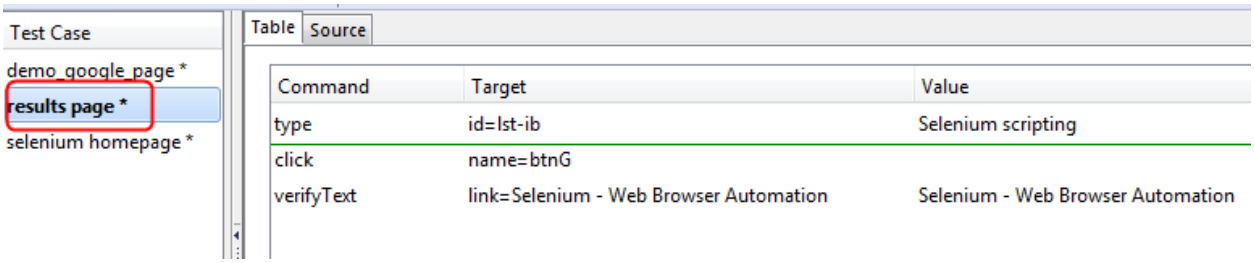


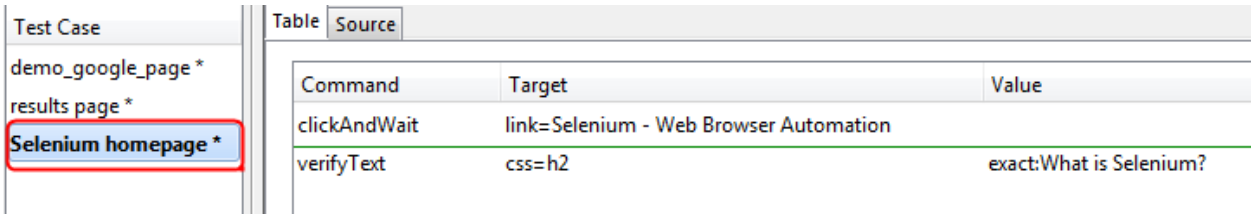
2) Rename the current "Untitled" test case and create 2 extra test cases to represent the 3 pages that the probe accesses.



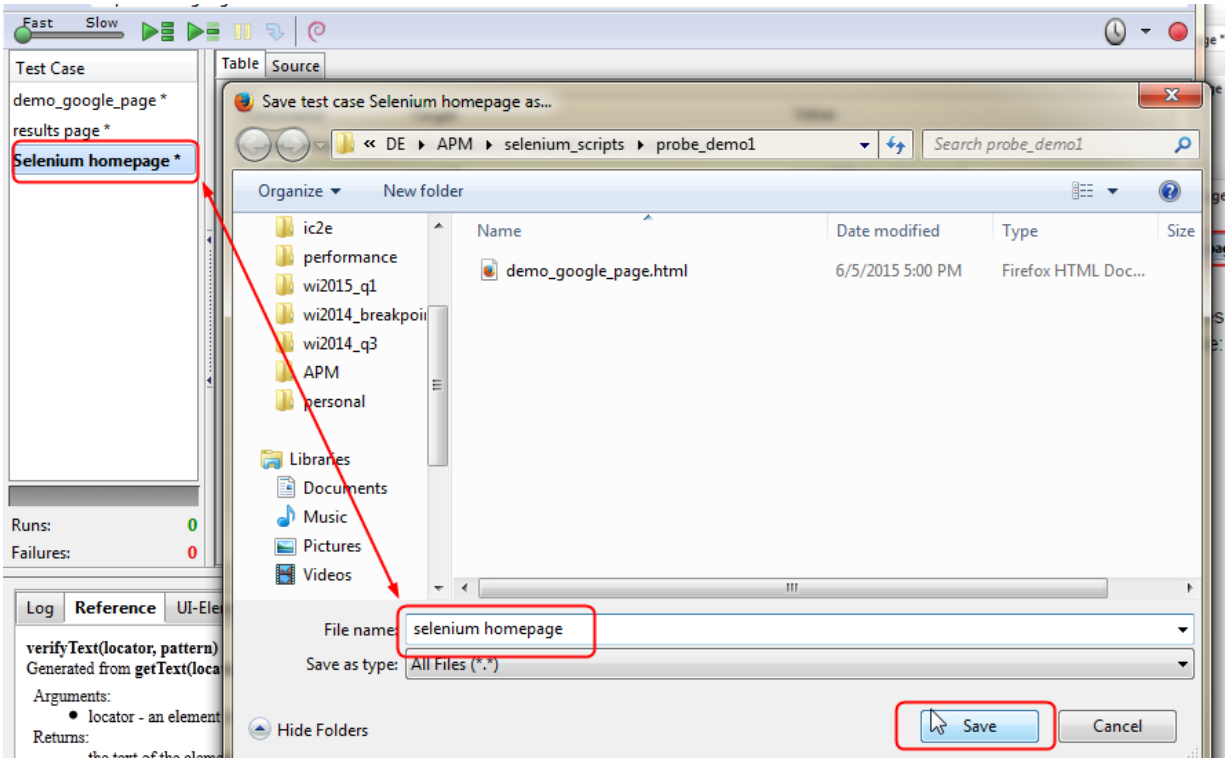
3) By using cut and paste, redistribute the commands to where they each belong. Remember, the **verifyText** command stays on the same test case page as the command that opens it. At the end you should have these 3 test cases containing these commands.





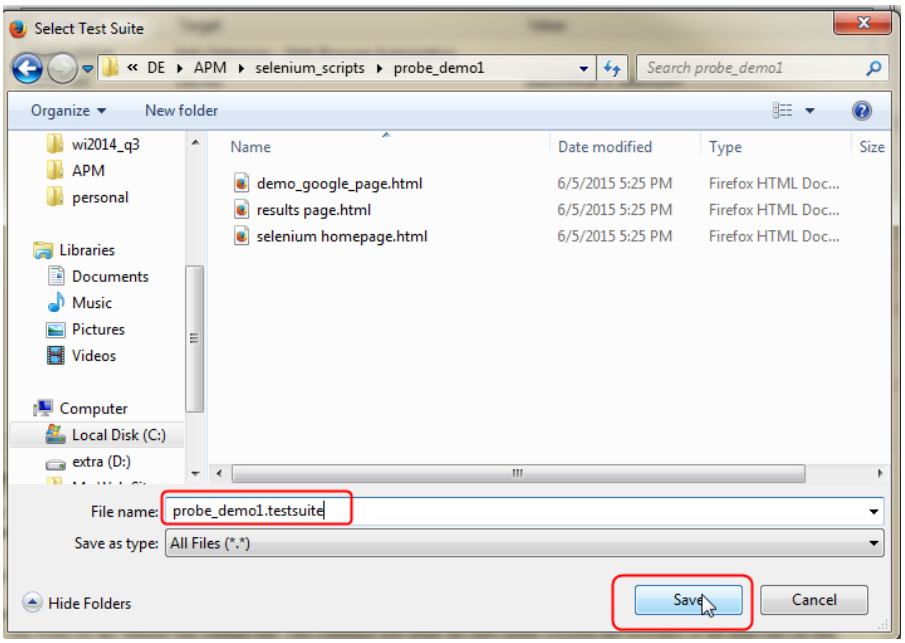


4) Save each test case (page) into its own html file. Ideally use the test case name as the html file name.



5) Last but not least, save this probing scenario as one **test suite**. For ease of loading into the Synthetic Script Manager afterwards, please give the test suite a ".testsuite"

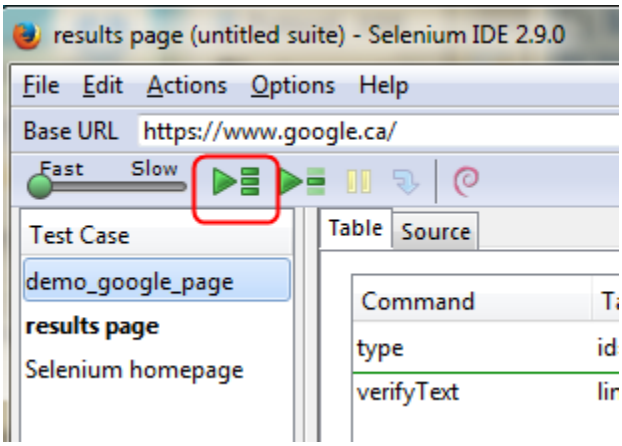
extension.



2.3 Testing the recorded script

Now that you have rearranged and saved the test case pages, it is time to test your script by running it in Selenium IDE.

1)



2) Verify the results by going to the logs tab. You may find the following error:

```
Log | Reference | UI-Element | Rollup
[info] Playing test case demo_google_page
[info] Executing: |open | /?gws_rd=ssl | |
[info] Test case passed
[info] Playing test case results page
[info] Executing: |type | id=lst-ib | Selenium scripting |
[info] Executing: |click | name=btnG | |
[info] Executing: |verifyText | link=Selenium - Web Browser Automation | Selenium - Web Browser Automation |
[error] Element link=Selenium - Web Browser Automation not found
[info] Test case failed
[info] Playing test case selenium homepage
[info] Executing: |clickAndWait | link=Selenium - Web Browser Automation | |
[error] Element link=Selenium - Web Browser Automation not found
[info] Test case failed
```

The reason that you see the failure is with Google, when you press on the search button, it doesn't go to a new page and Selenium's 'click' command does not know that. To resolve this, you will need to add a manual 'pause' command before the verifyText. Make sure you add the time required to pause in the target field, as shown in the next graphic.

Command	Target	Value
type	id=lst-ib	Selenium scripting
click	name=btnG	
pause	5000	
verifyText	link=Selenium - Web Browser Automation	Selenium - Web Browser Automation

3) Save and rerun the test suite. It should pass this time.

3 ADDING FLOW CONTROL TO THE SCRIPTS

There might be times where you want to have more control over the flow of the script. For example, when a 'shopping' probe logs in to a site and start shopping, it might expect the cart to be empty and there might be some dependencies on that for the script to work. In this scenario, you need to add the following logic to your script:

if cart is empty
    go to *continue*
else
    empty the cart
continue

With the flow control package, you can now do just that!

3.1 Download the Flow Control Firefox add-on

Go to this link and install the Selenium Flow Control add-on: https://addons.mozilla.org/en-us/firefox/addon/flow-control/

After this add-on is installed, the following commands are added:

gotoIf, gotolabel, while, label

These commands provide flow control in your selenium scripting.

3.2 Adding Flow Control to the script

Now let's take a look at how we can implement something as simple as the check for empty shop cart mentioned in the beginning of section 3:

gotoIf	"${total}"=="Total: 0 USD."	nothingincart
open	/isc/procurement/gcf/ibm5/opc//shoppingCart/show?backUrl...	
click	id=product-list-select-all	
clickAndWait	id=deleteSelectedItems	
assertConfirmation	Do you really want to delete the selected items?	
open	/isc/procurement/gcf/ibm5/opc/oci/index?cmd=OCILogin&c...	
label	nothingincart	
assertText	link=Cart (0 items, 0 USD)	Cart (0 items, 0 USD)
//select product and add to cart		

1) gotoIf here checks if the variable ${total} is 0 in the cart. If it is, then skip over commands and go directly to where it is labelled "nothingincart"
2) This tells the script to come to this line when (labelled "nothingincart") the cart is empty

4 USING VARIABLES IN YOUR SELENIUM SCRIPT

You can also use variables in Selenium IDE. Here are two examples where variables are used in Selenium scripts.

4.1 FOR loop

With a variable acting as a counter and gotoIf, you can create a for loop as shown below:

1) create a variable named 'counter' with the 'store' command:

Command	store
Target	0
Value	counter

2) then by using together with the **gotoIf/label** commands from the flow control add-on. You have a for loop:

Command	Target	Value
label	again	
storeEval	${counter}+1	counter
//exit loop and logout if refreshed more than 10 times		
gotoIf	${counter}>=10	very_end
//pausing 10 seconds		
pause	10000	
clickAndWait	//input[@value='Refresh']	
storeElementPresent	//td[@id='fullContent']/div/form/table[2]/tb...	isDone
//check if 18th row exist in table (workflow is done) if not go back to try 'again'		
gotoIf	storedVars['isDone']	Done
gotolabel	again	

Note that this loop starts from **label** → "again" and loops around until the **counter** variable reaches 10. Then it breaks to go to the **label** → "very_end".

4.2 Return variable

Some commands also return 'true' or 'false'. With Selenium IDE, you can store the returned value into a variable to be used later:

Command	Target	Value
storeElementPresent	//td[@id='fullContent']/div/form/table[2]/tbody/tr[18]/td	isDone
//check if 18th row exist in table (workflow is done) if not go back to try 'again'		
gotoIf	storedVars['isDone']	Done

1) This script checks if an element exists or not (in this case, the 18th row in a table on the page). The script then stores whether the value is true or false in the variable **isDone**.
2) The variable **isDone** is then checked before going to another section of the script (i.e. labelled Done)